Online Satellite Hitting Set

Spatial Grid Hashing + Multi-Objective Heuristic Scoring

Team Name : Optimus Prime

Mathematical Formulation

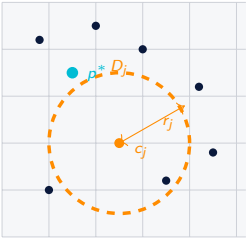
Problem Statement

Given satellites $S = \{p_1, \ldots, p_N\}$, $p_i \in \mathbb{R}^2$, answer Q online queries:

$$q_j = (c_j, r_j) \quad \Rightarrow \quad D_j = \{p \mid \|p - c_j\| \leq r_j\}$$

Goal: Output one $p^* \in D_j \cap S$ per query

- If $\exists$ active $p \in D_j \Rightarrow$ reuse it.
- Else activate and output the **best** $p^* \in D_j$.



Satellite Field

Formal Objective

$$\min |\mathcal{A}| = \left| \bigcup_{j=1}^{Q} \text{chosen}(D_j) \right|$$

Minimize total activated satellites. (NP-Hard offline $\Rightarrow$ heuristic online)

Spatial Data Structure: Multi-Scale Grid Hashing

Motivation

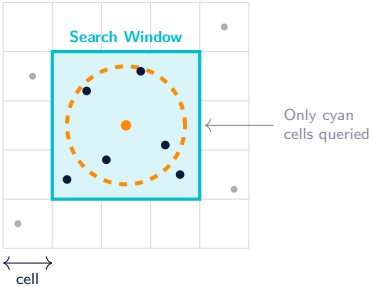
Brute-force: $\mathcal{O}(N)$ per query — too slow for $N, Q \leq 10^5$.

Solution: Two-level spatial hash partitions $\mathbb{R}^2$ into cells.

Cell Size Selection

$$\text{cell}_{\text{small}} = 25\,\text{u}, \quad \text{cell}_{\text{large}} = 100\,\text{u}$$

use small grid if $r < 50$, else large



Query procedure:

1. Compute grid cell of c_j
2. Expand by $\lceil r/\text{cell} \rceil + 1$ in each direction
3. Filter: $\|p - c_j\|^2 \leq r^2 + \varepsilon$

cells checked $= \mathcal{O}\left(\frac{r}{\text{cell}}\right)^2 \ll N$

$k(c_x, c_y) = (c_x + 50000) \cdot 200003 + (c_y + 50000)$

Hash map lookup: $\mathcal{O}(1)$ amortized per cell.

Heuristic Strategy: Multi-Objective Scoring

For each inactive candidate $p \in D_j$:

$$\boxed{\text{score}(p) = w_d\,\mathcal{C}(p) + w_c\,\mathcal{V}(p) + w_s\,\mathcal{I}(p)}$$

Centrality $w_d = 0.45$

$\mathcal{C}(p) = \dfrac{r_j - \|p - c_j\|}{r_j}$ Central points survive more future overlapping disks.

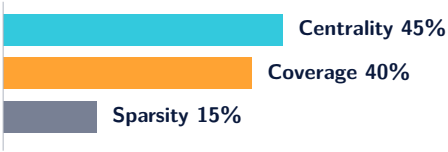
Coverage $w_c = 0.40$

$\mathcal{V}(p) = \dfrac{|\mathcal{N}(p, \min(100, 2r_j))|}{150}$ Dense neighborhoods imply high future reuse.

Sparsity $w_s = 0.15$

$\mathcal{I}(p) = 1 - \dfrac{\text{density}(p, 50)}{\text{maxDensity}}$ Rewards isolated points for

Weight Rationale



Intuition:

- **Central** $\Rightarrow$ survives shifted future disks
- **Dense zone** $\Rightarrow$ one sat covers many queries
- **Sparse bonus** $\Rightarrow$ avoids redundant clustering

NP-Hard Core

No poly-time optimal offline algorithm exists. Heuristics maximise *expected* future reuse.

Performance Optimizations

Fast Path: Active Satellite Check

Before any grid lookup, scan `active_list`:

- Active sat hits $D_j \Rightarrow$ output immediately.
- Overlapping queries trigger this frequently.

$\text{cost} = \mathcal{O}(|\mathcal{A}|) \ll \mathcal{O}(N)$

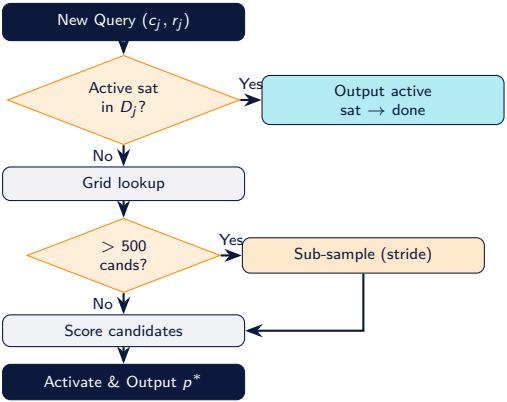
Precomputed Density

At init, for every p_i: $\text{density}(p_i) = |\mathcal{N}(p_i, 50)|$, cap$= 300$.

Amortises cost across all queries.

Sub-sampling for Dense Regions

If $|\text{cands}| > 500$: $\text{step} = \lfloor |\text{cands}|/500 \rfloor$. Uniform stride gives representative evaluation without TLE.



Time Complexity Analysis

Operation	Complexity	Notes
Build grids	$\mathcal{O}(N)$	One hash-insert per satellite
Precompute density	$\mathcal{O}(N \cdot K)$	$K \leq 300$ cap (early exit)
Fast-path check	$\mathcal{O}(\lvert\mathcal{A}\rvert)$	$\lvert\mathcal{A}\rvert \leq Q$, typically small
Grid query	$\mathcal{O}\left(\frac{r^2}{\text{cell}^2} + \lvert\text{cands}\rvert\right)$	$\ll \mathcal{O}(N)$
Scoring (dense)	$\mathcal{O}(500)$	Sub-sampled, constant
Scoring (sparse)	$\mathcal{O}(\lvert\text{cands}\rvert \cdot K')$	$K' \leq 150$ cap
Per-query total	$\mathcal{O}(\text{cells} + \min(\lvert\text{cands}\rvert, 500))$	**Sub-linear in N**

Grid vs. Brute Force

$$\underbrace{\mathcal{O}(N \cdot Q)}_{\text{brute force}} \longrightarrow \underbrace{\mathcal{O}\left(Q \cdot \frac{r^2}{\text{cell}^2}\right)}_{\text{grid hash}}$$

For $r = 50$, cell$= 25$: factor-of-N speedup.

Early-Exit Caps Prevent TLE

Density count capped at $K = 300$. Even with $N = 10^5$ co-located satellites, each `countNeighbors` call terminates in $\mathcal{O}(K)$.

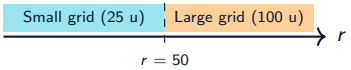
Edge Cases & Robustness

Floating-Point Safety

All distance checks use: $\|p - c_j\|^2 \leq r^2 + \varepsilon$, $\varepsilon = 10^{-9}$.

Prevents boundary satellites being wrongly excluded.

Scale-Adaptive Grid Selection



Empty Candidate Fallback

If grid returns no candidates, outputs satellite 0 rather than undefined behaviour.

Coverage Mode Switching

- $|\text{cands}| \leq 200$ **(sparse)**: full `countNeighbors`$(p, 2r, 150)$ for accurate coverage score.
- $|\text{cands}| > 200$ **(dense)**: use precomputed density; skip expensive recount.

I/O Optimisation for Interactive Judge

```
ios::sync_with_stdio(false);
cin.tie(nullptr);
// mandatory flush per reply:
cout<<chosen<<endl;
```

Unbuffered I/O + `endl` flush satisfies the interactive judge protocol.

Solution Summary

Mathematical Model

- Online hitting set on $\mathbb{R}^2$
- Disk queries (c_j, r_j)
- Score balances centrality, coverage, sparsity
- NP-Hard offline $\rightarrow$ greedy heuristic

Heuristic Strategy

- 3-objective weighted score
- w_d=0.45, w_c=0.40, w_s=0.15
- Active-list fast path
- Dense / sparse mode switch

Time Complexity

- Multi-scale grid hash
- $\mathcal{O}(1)$ cell lookup
- Sub-linear per query
- Caps prevent TLE

Spatial Hashing + Multi-Objective Greedy = Fast & Intelligent Satellite Selection